Standardising Open Source POIs labels with LLMs

Ivann Schlosser

Table of contents

Introduction

In this post, we will look into an application of LLMs, in particular text embedding, to reclassify the set of common open source points of interest (POIs) data sets into the type of economic activity they represent. This kind of data engineering can prove useful if you are studying local or regional economic activity profiles and specialisation across the diverse range of economic activity that the POIs indirectly describe. The spatial granularity of such data in particular makes it appealing for a data scientist looking at spatial profiles of economic activity. We will quickly see the strengths, but also limitations of this data.

Classifications of economic activity

In an effort to understand the global economy, several standards have been developed to describe all possible types of economic activity at different level of detail, producing a hierarchical tree of categories that are nested inside each other as the specificity of an individual label gets more complex. Several such standards exist, such as the NAICs or ISIC, and they evolve other time as well as some categories get appended or modified. While they are quite similar to each other, certain differences exist, and to keep things simple here, we will only use the ISIC in this post. The details of this classification can be found of the official website of the International Labor

Organisation. The NAICs classification details, mainly relevant for the United States, can be found here

Let's look at a specific example in the table, in growing order of detail:

Section	Division	Description
G	-	Wholesale and retail trade; repair of motor vehicles and motorcycles
G	47	Retail trade, except of motor vehicles and motorcycles
G	475	Retail sale of other household equipment in specialized stores
G	4751	Retail sale of textiles in specialized stores

As we can see, with every row, the description gets more specific, and the *division* level increases. The section remains the same because we are in the same broad *section* of economic activity: *wholesale and retail trade.* There are several different sections, such as agriculture, forestry, fishing (A), construction (F), Accomodation and food services (I) etc…
We can already see the link with the POIs here.

Data

Several sources of open source POIs are out there, we will look at 2 most popular and comprehensive ones, the Overture Foundation and Foursquare data sets. The provide global coverage across a very diverse range of categories.

POIs

Let us read all the available categories from the github repository for Overture POIs:

```
import os
import pandas as pd
import geopandas as gpd
```

```
overture_places_types = "https://raw.githubusercontent.com/OvertureMaps/schema/refs/heads/ma

places_types = pd.read_csv(
    overture_places_types,
    sep="; ",
    dtype={"Category code": str, " Overture Taxonomy": list},
    engine="python",
)

places_types.head(3)
```

	Category code	Overture Taxonomy
0	eat_and_drink	[eat_and_drink]
1	restaurant	[eat_and_drink,restaurant]
2	afghan_restaurant	[eat_and_drink,restaurant,afghan_restaurant]

In it's current form, the table is not so usable, let us do some transformations before proceeding further.

```
# rename columns
places_types.rename(
    columns={"Category code": "category", "Overture Taxonomy": "taxonomy"}, inplace=True
)

# strip and split the nested list
places_types["taxonomy"] = places_types["taxonomy"].apply(
    lambda x: x.strip().replace("]", "").replace("[", "").split(",")
)

# separate the nested labels into 3 columns
places_types["main_cat"] = places_types["taxonomy"].apply(lambda x: x[0])
places_types["sec_cat"] = places_types["taxonomy"].apply(
    lambda x: x[1] if len(x) > 1 else x[0]
)
places_types["raw_cat"] = places_types["taxonomy"].apply(lambda x: x[len(x) - 1])

places_types.sample(3)
```

	category	taxonomy	main_cat
511	baseball_field	[active_life, sports_and_recreation_venue, bas...	active_life
1856	appellate_practice_lawyers	[professional_services, lawyer, appellate_prac...	professional_services
2102	dam	[structure_and_geography, dam]	structure_and_geogr

We observe here that the labels are also organised in a nested structure that can be divided into 3 levels that we will call `main_cat`, `sec_cat`, `raw_cat`. This is now in a more usable format.

ISIC

Let us get back to the economic activity classification, to get straight to the point, we will skip some data engineering done on this data and provide it:

```
import os

print(os.getcwd())
isic_double = pd.read_csv("./data/isic_double_dense.csv", index_col=0).reset_index(
    inplace=False, drop=True
)

isic_double.sample(5)
```

```
/Users/cenv1069/Documents/quarto_blog/posts/econ_pois
```

	Code	Description	section	detailed_descr
42	47	Retail trade, except of motor vehicles and mot...	G	Retail trade, except of motor vehicles
24	26	Manufacture of computer, electronic and optica...	C	Manufacture of computer, electronic
39	43	Specialized construction activities	F	Specialized construction activities, D
75	86	Human health activities	Q	Human health activities, Hospital ac
23	25	Manufacture of fabricated metal products, exce...	C	Manufacture of fabricated metal proc

In this table, I provide levels of the ISIC classification up to a 2 digit division resolution. This means that the description for the finer scale labels were all joined together. We will actually simplify this even more as you will see. You can now get a better idea of the labels by reading the detailed description that was also merged with the simple labels, this will be important for us down the line.

Methods

We now get a better picture of the challenge, we have two relatively complex semantic structures representing categorical variables, on the one hand labels of open source POIs, such as `restaurant`, or `hospital`, and on the other descriptions of economic activities, such as `Manufacture of chemicals and chemical products`, `Travel agency, tour operator, reservation services`, we would like to create a link between the two and come up with a mapping from one to the other. One way is to do it manually for all lables, but this is work intensive and would make up for a pretty boring blog post. So in order to automate and scale this workflow and make use of some cool technoloogy, let us turn towards Large Language Models (LLMs). There is no need to introduce such tools as ChatGPT, Claude or Gemini these days as they have now integrated so many aspects of our life and work. This post is in no way aimed at introducing the complex macinery behind the 'magic' of LLMs, but rather to apply it in our specific use case. Let us still have a quick look into what we are doing here, for that let us decompose the GPT in Chat*GPT*. It stands for Generative Pretrained Transofrmer, the Generative part is less relevant for us, but the Pretrained Transformer is crucial for addressing our problem. LLMs, as it turns out, can be seen as very complex compression algorithms, that have seen extensive amounts of texts and patterns in text, and through training come up with a simplified, compressed, or *transformed* representation in a high dimensional vector space. An empty LLM is just a set of randomly computed matrices, which will generate gibberish, but a *pretrained* one is one that has been shown enough text data to be able to construct a transformed representation of a body of literature that can in turn be used to analyse and even generate text. In this transfomed representation, more related words, or phrases, will be mapped closer to each other in the high dimensional vector space. This is important for us because we want to map related, but different things to each other in our two catogory data sets.

So with this small introduction, we can now construct a small workflow that will take advantage of LLMs to transform the categorical lables, producing what is called embeddings, that we can then map to each other based on cosine similarity, a very common metric to compare two vectors.

For that, we will use a python library providing access to a familly of open source light weight LLMs called SBERT. They are light weight, so we can run the models locally on our machine and they are open source, hosted in a Hugging Faces repo, which means their weights are publicly availble, a crucial aspect for open research.

Let us write some support functions for the next steps:

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity


### Embedding and matching function
```

```
def embedding_match_category(category, reference_data):

    # encode the provided categoriess
    poi_embedding = model.encode(category)

    # get the precomputed embeddings from reference data
    embeddings = list(reference_data["embedding"])

    # compute cosine similarities between the embeddings of the provided categories and the r
    similarities = cosine_similarity([poi_embedding], embeddings)[0]

    # find the best scores
    best_match_index = similarities.argmax()

    # retrived the corresponding best match for every input category.
    best_match_score = similarities[best_match_index]
    best_match_row = reference_data.iloc[best_match_index]

    # return the values and score
    return [
        best_match_row["section"],
        best_match_row["Code"],
        best_match_row["Description"],
        best_match_score,
    ]
```

The above function implemented the steps we describe earlier. Let us now select a model to use, and apply it to the `sec_cat` variable of our labels:

```
selected_model = "all-mpnet-base-v2"
model = SentenceTransformer(selected_model)

# calculate embedding for the ISIC classifications using detailed descriptions for richer co
isic_double["embedding"] = isic_double["detailed_descr"].apply(
    lambda x: model.encode(x)
)

# match with the overture secondary labels
places_types[["section", "isic_embed", "isic_descr", "match_score"]] = [
    *places_types["sec_cat"].apply(
        lambda x: embedding_match_category(x.replace("_", " "), isic_double)
```

```
    )
]
```

```
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
```

We now have a table of overture categories with their best textual matches from the ISIC classification based on the chosen LLM model to perform the embedding. This, of course, can not be considered a absolute truth and a manual sense checking and correction is mandatory, especially since the size of the data is not so big (~ 700 labels). Let us inspect some of it:

```
places_types[["sec_cat", "section", "isic_descr"]].sample(5)
```

	sec_cat	section	isic_descr
60	restaurant	I	Food and beverage service activities
635	eyelash_service	S	Other personal service activities
505	sports_and_recreation_venue	R	Sports activities and amusement and recreation...
13	restaurant	I	Food and beverage service activities
252	automotive_dealer	G	Wholesale and retail trade and repair of motor...

Application

Let us now explore a simple application of the recategorised data, we will first download some POIs for the city of Paris:

```
bbox = [
    2.15,
    48.73,
    2.6,
    49.00,
]

bbox_str = [str(val) for val in bbox]

places_filename = "data/places.geoparquet"

if not os.path.exists("data/places.geoparquet"):
    try:
        os.system(
            f"overturemaps download -f geoparquet --bbox={','.join(bbox_str)} --type=place -
        )
```

```
    except Exception as e:
        print(e)

places = gpd.read_parquet(places_filename)

places.head(3)
```

	id	geometry	categories
0	3cbf89f8-989c-4230-b761-4e63289fc40d	POINT (2.18249 48.73138)	{'primary': 'restaurant', 'alternate':
1	09240397-659e-4765-becc-6a2a09ed874d	POINT (2.17989 48.74051)	{'primary': 'garbage_collection_ser
2	8bd25885-8f7b-42d8-bbaa-12100fe8970d	POINT (2.18003 48.73668)	{'primary': 'clothing_store', 'altern

Now we can merge our ISIC mapping to the primary category after unnesting this value.

```
places["primary"] = places["categories"].apply(
    lambda val: val["primary"] if val is not None else ""
)

places = places.merge(places_types, left_on="primary", right_on="raw_cat", how="left")

places.head()
```

	id	geometry	categories
0	3cbf89f8-989c-4230-b761-4e63289fc40d	POINT (2.18249 48.73138)	{'primary': 'restaurant', 'alternate'
1	09240397-659e-4765-becc-6a2a09ed874d	POINT (2.17989 48.74051)	{'primary': 'garbage_collection_se
2	8bd25885-8f7b-42d8-bbaa-12100fe8970d	POINT (2.18003 48.73668)	{'primary': 'clothing_store', 'altern
3	f779dc3c-2995-464e-85ef-dcb34d1a5462	POINT (2.18214 48.73057)	{'primary': 'farm', 'alternate': ['fru
4	7b7047ad-7c74-4298-a3c9-c9d6bb952983	POINT (2.18108 48.731)	{'primary': 'automotive_repair', 'a

We merge on the `raw_cat` variable as it's the one corresponding to the raw data. Let us now visualise the the spatial distribution of economic activity in Paris. To do this, we will first project all the POIs onto the H3 grid. If you are not familiar with this spatial index, have a look at the blog post and look at the official documentation of this API.

```
import h3


h3_res = 8
```

```
places["h3_id"] = places["geometry"].apply(
    lambda geom: h3.latlng_to_cell(geom.y, geom.x, h3_res)
)

sector_density = (
    places[["h3_id", "section"]]
    .value_counts(subset=["h3_id", "section"])
    .reset_index(inplace=False, drop=False)
    .drop_duplicates("h3_id")
)
sector_density.head()
```

	h3_id	section	count
0	881fb46665fffff	G	909
1	881fb46621fffff	G	907
2	881fb4662dfffff	G	883
3	881fb475b5fffff	G	868
5	881fb46667fffff	I	733

Now we add cell geometries and set up some plotting parameters.

```
import shapely as shp
import pypalettes as pypal

sector_density["geometry"] = sector_density["h3_id"].apply(
    lambda val: shp.geometry.shape(h3.cells_to_geo([val]))
)

cat_palette = pypal.load_cmap(["Joyful", "Juarez", "Kandinsky"])

sector_gdf = gpd.GeoDataFrame(sector_density, geometry="geometry", crs="epsg:4326")
```

And plot:

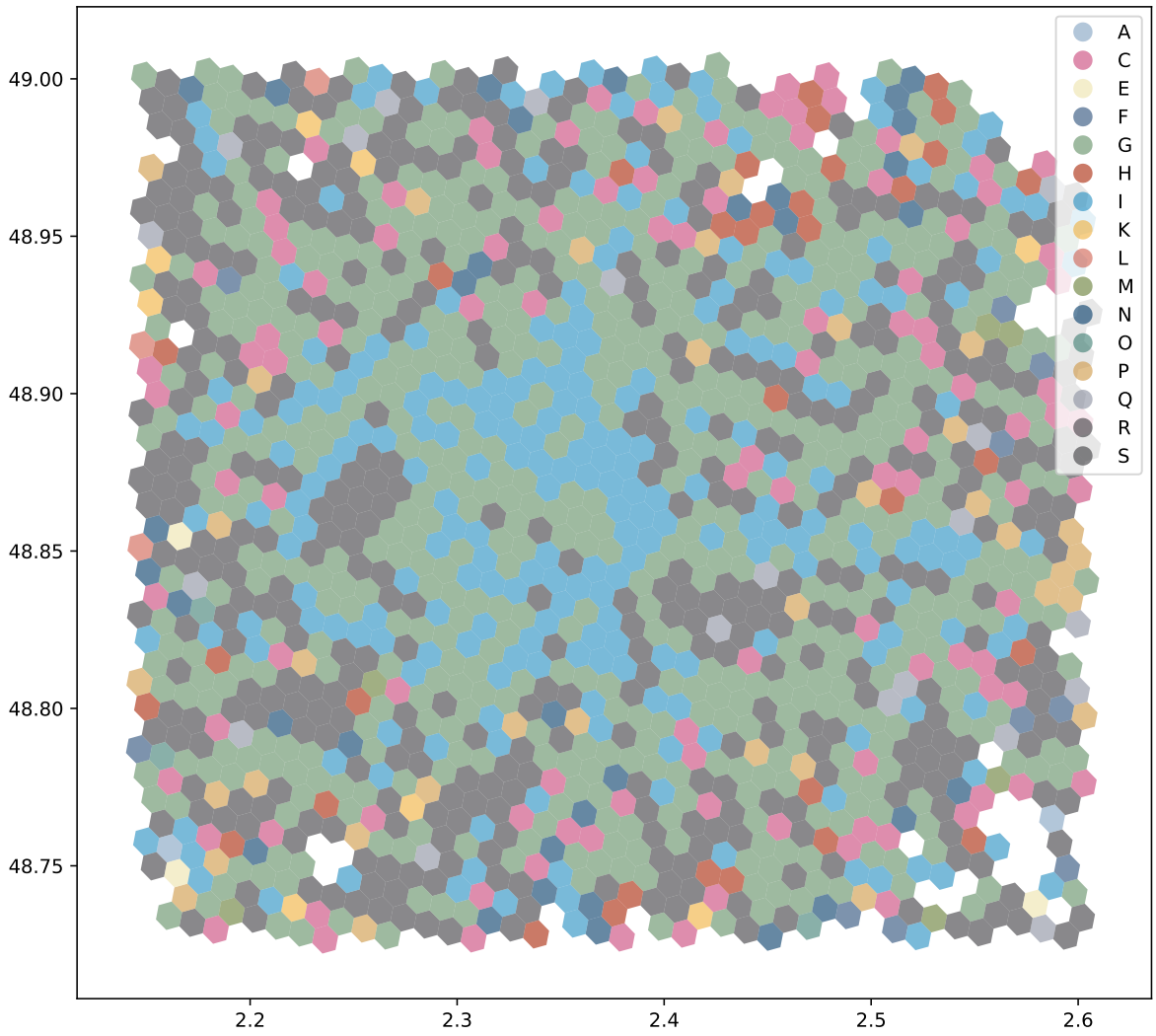
```
import matplotlib.pyplot as plt
import contextily as cx

fig, ax = plt.subplots(figsize=(10, 10))
```

```
sector_gdf.plot(
    ax=ax,
    column="section",
    cmap=cat_palette,
    legend=True,
    alpha=0.6,
)

# cx.add_basemap(ax, crs="epsg:4326", source=cx.providers.Stamen.Toner)

plt.show()
```



We see sections